MODULE 0 · FOUNDATIONS

Cluster Networking & the CNI

Every Pod gets an IP — a plugin makes that promise real

WHY IT MATTERS

Kubernetes defines the network model — it ships none of it

Kubernetes specifies the rules a cluster network must satisfy, but the **cluster itself doesn't implement them**. That gap is filled by a **CNI plugin** (Container Network Interface).

Every node's `Ready` status, every direct Pod-to-Pod connection, and every working `ClusterIP` all depend on this layer being wired in correctly.

BY THE END YOU CAN

- State the four rules of the Pod network model
- Explain what a CNI plugin does at Pod creation
- Compare kube-proxy modes
- Recognise a missing-CNI or CIDR-overlap failure

CONCEPT

Four rules define a flat network

- **Every Pod gets its own IP** — a real, cluster-routable address, not a port on the node.
- **Pods talk to Pods without NAT** — any Pod on any node reaches any other Pod directly.
- **Node agents reach local Pods** — the kubelet and system daemons reach Pods on their own node.
- **A Pod sees its own IP** — the address it thinks it has is the address everyone else sees.

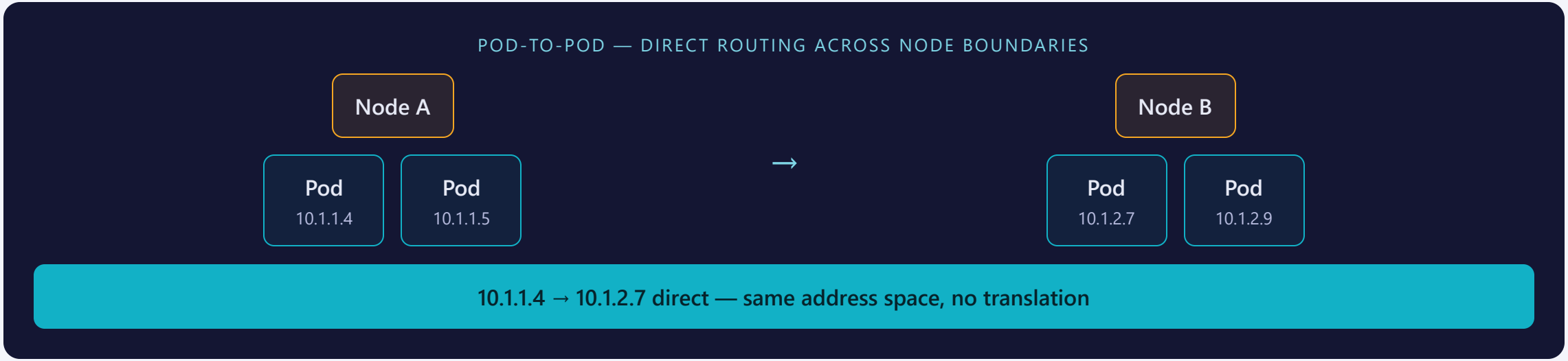
THE RESULT

One address space. No NAT between Pods, on any node.

The CNI plugin's only job is to make this true no matter how many nodes the cluster has.

THE FLAT NETWORK

Any Pod, any node, no NAT

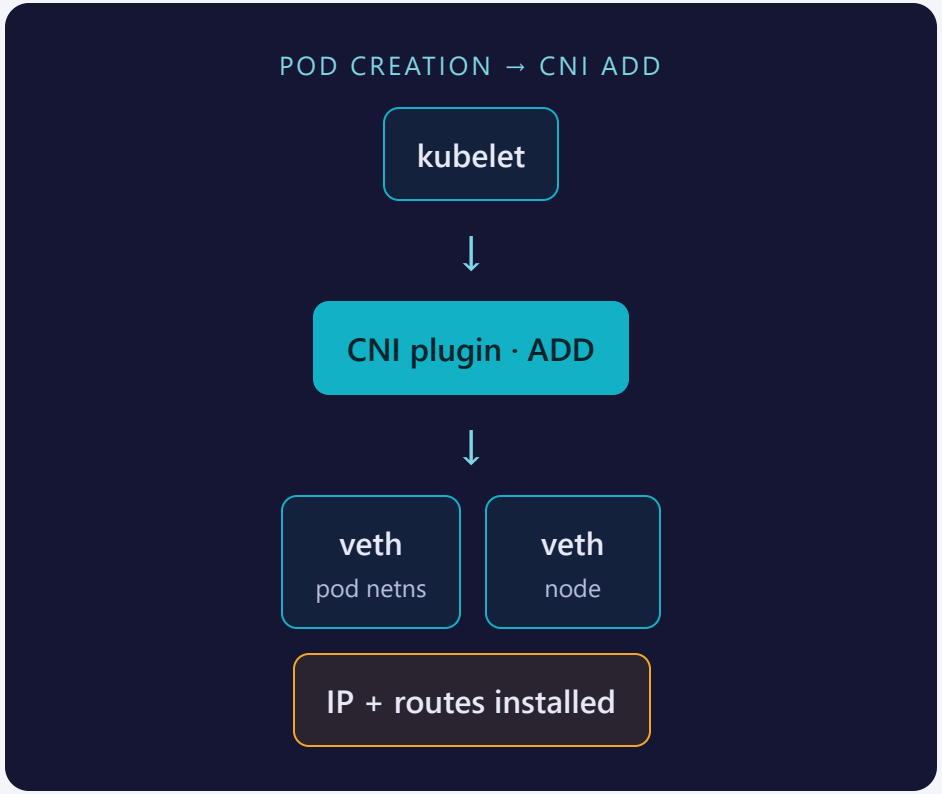


AT POD CREATION

The kubelet calls the plugin; the plugin wires the netns

When a Pod is scheduled, the kubelet invokes the CNI plugin's `ADD` command. The plugin then:

- Creates a **veth pair** — one end inside the Pod's network namespace, the other on the node
- Assigns the Pod an **IP from the node's pod CIDR** slice
- Installs the **routes** so that IP is reachable from the rest of the cluster



PLUGINS VARY, THE CONTRACT DOESN'T

Different approaches, same four rules

APPROACH	TYPICAL EXAMPLE	ENFORCES NETWORKPOLICY?
Overlay (encapsulated)	e.g. a VXLAN-based plugin such as Flannel	Often no — accepts the object, doesn't enforce it
Native routing	e.g. a BGP/IP-in-IP plugin such as Calico	Yes
eBPF dataplane	e.g. an eBPF-based plugin such as Cilium	Yes — can also replace kube-proxy

ANY OF THESE SATISFY THE MODEL

The four rules don't dictate the mechanism — overlay, native routing, and eBPF are all valid ways to deliver them.

FROM VIRTUAL IP TO REAL POD

kube-proxy modes rewrite a ClusterIP to an Endpoint

MODE	MECHANISM	NOTES
`iptables`	Linux netfilter chains, random backend pick	Default; rule count grows with Service count
`IPVS`	In-kernel IPVS load balancer, hash lookup	Scales further than `iptables`
eBPF (kube-proxy replacement)	Some CNI plugins program the dataplane directly	Skips `iptables` /IPVS entirely

CLUSTERIP IS VIRTUAL

Nothing listens on a Service's ClusterIP. Whichever mode is active rewrites it to a real Pod IP taken from the Service's **Endpoints**.

GOTCHAS — THE ONES THAT BITE

Where people trip

- **No CNI → node `NotReady`.** A fresh node with no CNI never goes Ready and its Pods sit `Pending` — first thing to check on a stuck new node.
- **NetworkPolicy needs an enforcing CNI.** Apply a policy under a non-enforcing plugin and it's accepted but never enforced.
- **Overlapping CIDRs break routing.** The pod CIDR must not overlap the node network or the Service CIDR — overlap causes silent, baffling misroutes.
- **ClusterIP is not a real destination.** No Endpoints means no backend to rewrite to — the connection just fails.

RECAP

Cluster networking in one breath

- Kubernetes defines the network model; a **CNI plugin** implements it
- Four rules → a **flat network**: every Pod has a real IP, no NAT between Pods
- At Pod creation the plugin makes a **veth pair**, assigns an IP, installs routes
- kube-proxy (`iptables`/`IPVS`) or an eBPF dataplane turns a virtual **ClusterIP** into a real Pod IP

Next: **Storage & the CSI**